

Modelagem MongoDB – Embedding vs Referencing

FATEC Jacaréi · Dia 2: Modelagem MongoDB e Embedding vs Referencing · 11/05/26

1. Objetivo

Definir como os dados do sistema de gestão de leads da **1000 Valle Multimarcas** serão estruturados no MongoDB (**leads_db**), cobrindo: captação e qualificação de leads, vínculo com customer/store/attendant, negociação ativa, histórico de negociação, auditoria e indicadores.

2. Visão C4 – Contexto e Container

Contexto: atores do sistema

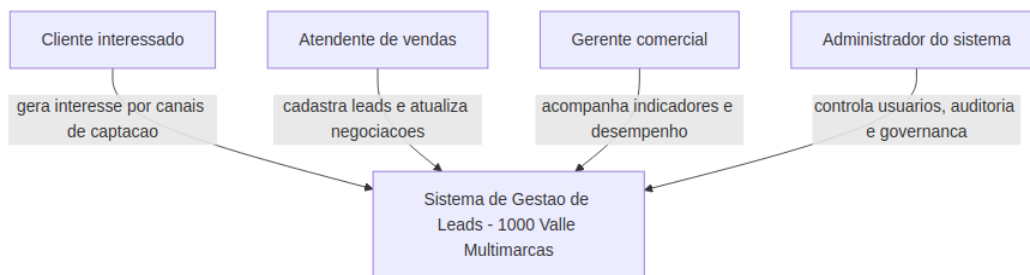


Figura 1 – Atores externos e sistema.

Container: stack tecnológico

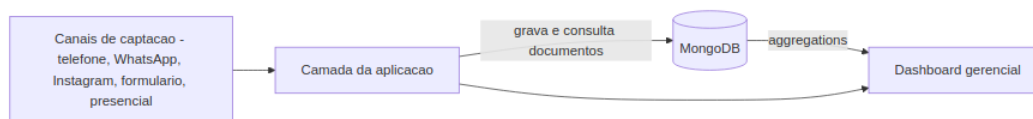


Figura 2 – Canais → Aplicação → MongoDB → Dashboard.

3. Coleções (leads_db)

users — admins, gerentes, atendentes

`_id, name, role, active`

stores — lojas/unidades

`_id, name, city, state`

customers — clientes

`_id, name, phone, createdBy`

leads — oportunidades

`_id, customerId, storeId, attendantId, channel, createdAt`

negotiations — processo comercial

`_id, leadId, importance, status, currentStage, history[ ]`

logs — auditoria

`_id, userId, action, entity, createdAt`

teams — equipes (reservada)

—

Referências entre coleções:

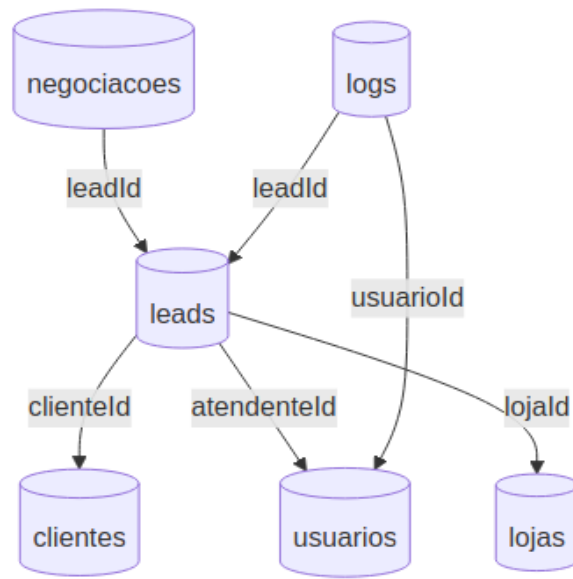


Figura 3 – Coleções e referências (C4 componentes).

4. Embedding vs Referencing

A decisão depende de: padrão de leitura, crescimento dos dados e independência da entidade.

Embedding — negotiations.history

O histórico pertence à negociação, é sempre lido junto com ela e não cresce de forma independente.

```
{
  leadId: ObjectId("..."),
  importance: "quente",
  status: "aberta",
  currentStage: "contato_inicial",
  history: [
    { stage: "contato_inicial", status: "aberta",
      changedAt: Date, changedBy: ObjectId }
  ]
}
```

Referencing — demais relacionamentos

- leads.customerId → customers (cliente pode ter vários leads)
- leads.attendantId → users (atendente cuida de muitos leads)
- leads.storeId → stores (loja recebe muitos leads)
- customers.createdBy → users
- negotiations.leadId → leads
- logs.userId → users (logs crescem muito; ficam separados)

5. Diagrama de Decisão

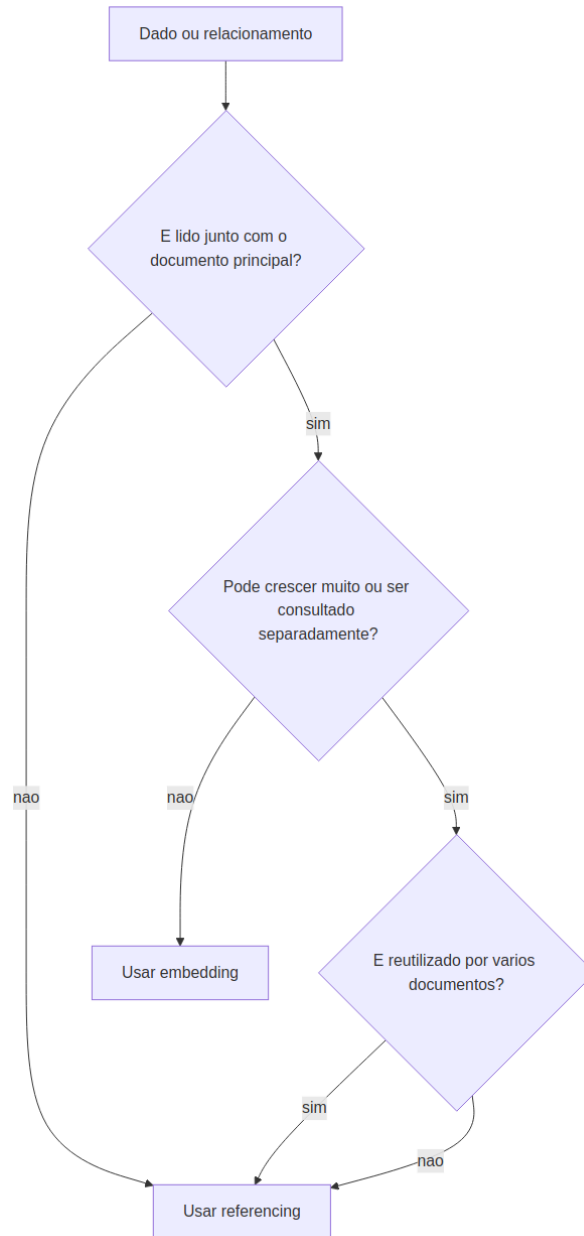


Figura 4 – Fluxo de decisão: embedding vs referencing.

6. Modelo ER

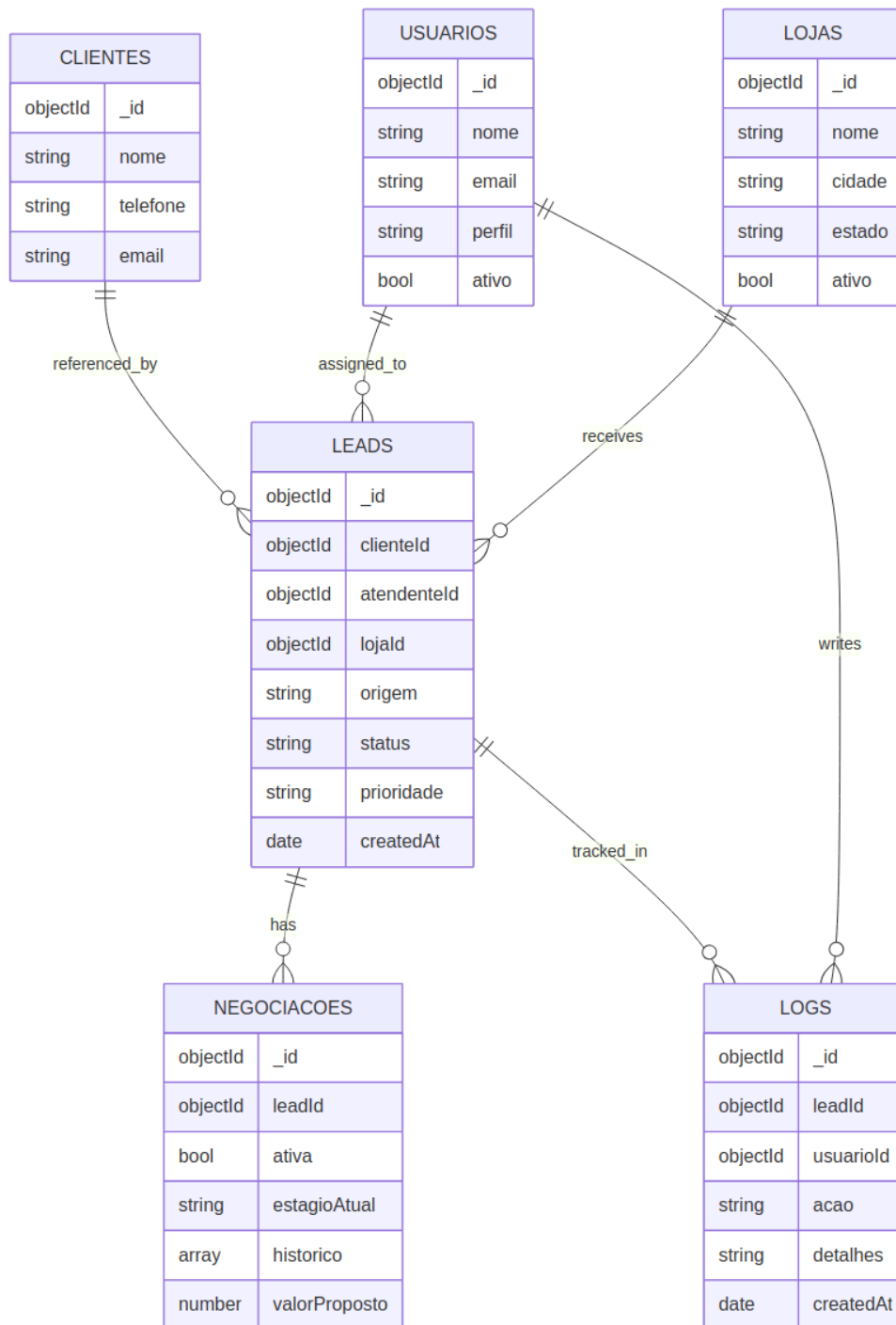


Figura 5 – Modelo ER completo orientado a MongoDB.

7. Fluxo Principal

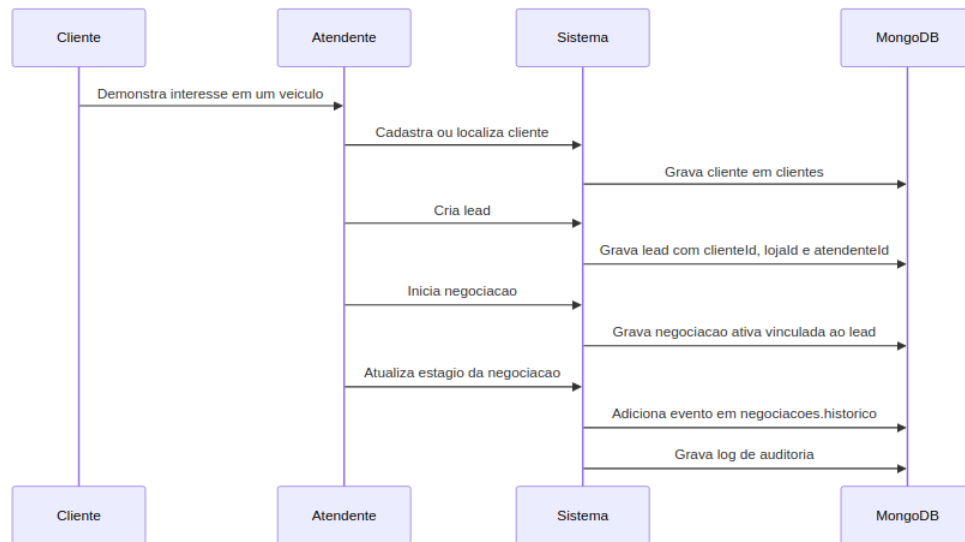


Figura 6 – Sequência: do interesse do cliente ao log de auditoria.

8. Regras Atendidas

- lead vinculado a customer (customerId), store (storeId) e attendant (attendantId)
- negotiations.leadId aponta para leads
- histórico por embedding em negotiations.history
- logs separados para auditoria
- apenas uma negociação ativa por lead — garantida pela aplicação

9. Índices Recomendados

```
db.leads.createIndex({ customerId: 1 });
db.leads.createIndex({ attendantId: 1 });
db.leads.createIndex({ storeId: 1 });
db.leads.createIndex({ channel: 1, createdAt: -1 });
db.negotiations.createIndex({ leadId: 1 });
db.logs.createIndex({ userId: 1, createdAt: -1 });
```

10. Conclusão

O modelo combina referencinɡ para entidades independentes (users, stores, customers) e embedding para o histórico interno de negotiations. Essa combinação mantém dados normalizados onde importa e performáticos onde a leitura é frequente.